Guía de examen (Parcial 2)

Instrucciones: Estudia los siguientes puntos. En el examen se te presentarán afirmaciones y deberás decidir si son Verdaderas o Falsas basándote en cómo funcionan estas herramientas.

1. GitHub Actions y los archivos YAML

- Los flujos de trabajo (Workflows) de GitHub Actions se definen en archivos con formato YAML (extensión. yml).
- Estos archivos se guardan dentro del repositorio.
- El flujo de trabajo no se ejecuta solo; se activa (se "dispara") cuando ocurre un evento en el repositorio (por ejemplo, un push o un pull request).
- Nota para el examen: Recuerda que la relación es: Archivo YAML -> se activa por un evento.

2. El comando git branch -M main

- No crea una rama nueva desde cero.
- Su función es renombrar la rama actual en la que estás trabajando.
- Por ejemplo, si estás en la rama master y ejecutas este comando, master desaparece y su historial pasa a llamarse main. La letra -M significa Move/Rename (Mover/Renombrar).
- Nota para el examen: Si una pregunta dice que este comando sirve para "crear" una rama, la respuesta es que eso es incorrecto.

3. Protección de la rama main (Branch Rulesets)

- Cuando proteges la rama main en GitHub (por ejemplo, exigiendo que haya un Pull Request antes de fusionar), estás bloqueando la escritura directa.
- Si intentas hacer un git push directo a main desde tu computadora, GitHub rechazará la subida y te mostrará un error (violación de reglas).
- Nota para el examen: No se puede hacer un push normal a una rama protegida sin pasar por un Pull Request.

4. El comando git checkout -b dev

- Es un comando que hace dos cosas al mismo tiempo: crea una nueva rama llamada dev y te cambia a ella automáticamente.
- La bandera -b significa branch (rama).
- Nota para el examen: Este comando sí cumple la función de crear y moverse a la nueva rama en un solo paso.

5. Solución de problemas en Workflows (Logs)

- Cuando un flujo de trabajo en GitHub Actions falla, la plataforma genera un registro detallado paso a paso.
- Lo correcto y recomendado ante un fallo es abrir la sección de Logs (registros de ejecución) para leer exactamente qué comando o línea de código causó el error.
- Nota para el examen: Volver a ejecutar el flujo de trabajo sin haber revisado los logs primero no es una buena práctica.

Parte 2

Instrucciones para el examen: El día del examen usarás Visual Studio Code. Deberás ejecutar los comandos en la terminal y tomar capturas de pantalla de cada paso. Al final, entregarás un PDF con las capturas y el enlace de tu repositorio (subirlo a classroom).

1. Preparación

- Abre Visual Studio Code en la carpeta de tu proyecto.
- Abre la terminal (Ctrl + ~).
- Si es la primera vez que usas Git en esa computadora, configura tu identidad:
    - git config --global user.name "Tu Nombre"
    - git config --global user.email "tu-correo@ejemplo.com"

- Inicializa el repositorio en esa carpeta:
    - git init

2. Primer Commit (Guardar cambios)

- Para ver qué archivos han cambiado o son nuevos:
    - git status (Los archivos en rojo son los que no están guardados).

- Para preparar todos los archivos para guardarlos:
    - git add. (El punto significa "todos").

- Para guardar el paquete de cambios con un mensaje:

    - git commit -m "Examen: Primer commit"

3. Subida a GitHub (El gran paso)

- Primero, ve a la página de GitHub y crea un repositorio público y vacío (sin README).
- Copia la URL de tu repositorio (ej. https://github.com/tu-usuario/tu-repo.git).
- Asegúrate de que tu rama local se llame main (GitHub lo pide así):
    - git branch -M main
- Conecta tu computadora con GitHub usando la URL que copiaste:
    - git remote add origin https://github.com/tu-usuario/tu-repo.git
- Sube tu código a la nube:
    - git push -u origin main

(NOTA IMPORTANTE: Si la terminal te pide "Username" pon tu usuario de GitHub. Si te pide "Password", NO pongas tu contraseña de GitHub, debes pegar tu Token (PAT)).

4. Crear una rama y modificarla

- Crea una nueva rama para trabajar (y cámbiate a ella automáticamente):
    - git checkout -b dev
- Abre cualquier archivo de tu proyecto y escribe una línea nueva (ej. "Este es un cambio en la rama dev").
- Revisa el estado (verás el archivo en rojo):
    - git status
- Guarda ese cambio:
    - git add .
- Haz el commit del cambio:
    - git commit -m "Examen: Cambio en rama dev"
- Sube esa rama nueva a GitHub:
    - git push origin dev